

Security lab notes

Started keeping this because I kept forgetting how I'd fixed the same three Docker/Terraform/Sigma problems the week before. It's not a tutorial, it's just what I actually did and what actually broke.

What's running right now, on my own Mac: Wazuh (self-hosted SIEM), TheHive + Cortex (SOAR), LocalStack (fake AWS), Suricata (already had this one), sigma-cli, gitleaks, Terraform. All free, all local except for the Sigma → KQL output which is meant for a real Sentinel workspace eventually.

Why bother with a local lab at all

A SIEM does three things really: pulls logs in from wherever they live, normalizes them into something you can actually compare across sources, and lets you search/alert on patterns in that normalized data. Sentinel does this as a paid cloud service with KQL. Wazuh does the same three things for free, with its own XML rule format and OpenSearch underneath instead. The actual concept transfers fine between them — the syntax doesn't.

Sigma exists because writing the same detection logic three times for three different query languages is a waste of time. Write the pattern once in Sigma's YAML format, then a converter turns it into KQL, or Lucene, or SPL, or whatever. The Sigma file is the actual asset — the KQL is just one compiled output of it, which is why I keep both in the repo rather than just the KQL.

ATT&CK is just a shared vocabulary for attacker behaviour — tactics (the goal), techniques (how), sometimes sub-techniques (the specific variant). Every rule I write gets tagged with the technique it's supposed to catch. That's also literally what interviewers ask about, so it's not optional to learn this part.

SOAR is the "now do something about it" layer that sits after the SIEM tells you something happened. TheHive is the case-management half, Cortex is the automation half — a Cortex analyzer is roughly the open-source version of one step in a Sentinel playbook.

Terraform: instead of clicking through the Azure portal (not repeatable, not reviewable, no history), you write down what you want and a tool reconciles reality to match it. ``plan`` shows the diff before anything happens, ``apply`` does it, ``destroy`` undoes it. Most mature security teams deploy their actual detection infrastructure this way now — "detection as code" isn't just a buzzword on a job posting, apparently.

Getting everything running

```
cd ~/securitylab/wazuh-docker/single-node && docker compose up -d
cd ~/securitylab/thehive && docker compose up -d
cd ~/securitylab/localstack && docker compose up -d
cd ~/securitylab/suricata && docker compose up -d

docker compose stop # keeps data
docker compose down # removes containers, keeps volumes
docker ps
docker system prune # if disk gets tight
```

Don't run everything at once unless you're actually using all of it — these stacks together ate through about 25GB of disk the first time I brought them all up, mostly because none of Wazuh/TheHive/Cortex have arm64 builds yet so they're all running under x86 emulation on my Mac. Slower to start than you'd expect, not actually broken.

Wazuh — first login

<https://localhost>, accept the self-signed cert warning (it's my own local CA, not an actual risk on localhost). Default login is admin/SecretPassword — change it immediately, it's published in their own docs so leaving it is the one genuinely bad habit this setup could teach. Threat Hunting and the other views are empty until there's a real log source feeding in, which there isn't yet — Suricata is the obvious next thing to wire in.

Reading a Sigma rule

Structure's straightforward: `logsource` says what kind of log it expects, `detection.selection` is the actual condition, `detection.condition` says how multiple named blocks combine, `falsepositives` is where I try to be honest about what else might trigger it, `level` is severity.

```
cd ~/security-portfolio/detection-engineering
export PATH="$HOME/.local/bin:$PATH"
make convert
cat kql-conversions/generated/impossible-travel.kql
```

TheHive — first case

http://localhost:9000. First run makes you set up the org and admin account yourself, no default login, which I appreciate even though it's one more step. Create a case, add an observable (even a fake IP from the 198.51.100.0/24 documentation range works fine for testing), that's the object Cortex would enrich once it's linked up.

LocalStack — first commands

```
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test
export AWS_DEFAULT_REGION=us-east-1

aws --endpoint-url=http://localhost:4566 s3 mb s3://my-lab-bucket
aws --endpoint-url=http://localhost:4566 iam create-user --user-name lab-user
```

Terraform — first apply

```
cd ~/security-portfolio/terraform-labs/01-first-resource-group
az login
cp terraform.tfvars.example terraform.tfvars # fill in your subscription id
terraform init
terraform plan
terraform apply
terraform destroy
```

Set the Azure spending cap before apply, not after. Wrote this down because it's exactly the kind of step I'd otherwise skip in the moment.

Making the lab actually do something

Suricata into Wazuh

Right now these run independently. Wiring them together: confirm Suricata's writing to `eve.json`, point a `<localfile>` block in Wazuh's `ossec.conf` at it (or mount the log volume into the manager container), restart the manager, check Threat Hunting for the test alert. This one's genuinely fiddly the first time — expect to burn real time on file paths and container networking before it clicks.

A custom Wazuh rule

```
<group name="local,suricata,">
  <rule id="100010" level="10">
    <if_sid>86601</if_sid>
    <field name="alert.severity">1</field>
    <description>High-severity Suricata alert</description>
  </rule>
</group>
```

Wazuh uses XML, not Sigma, for its own rules — Sigma just isn't a great fit for host/network telemetry the way it is for cloud logs, so I didn't force it.

New Sigma rules

Good next candidates from the sign-in logsource I already have working: T1110 brute force (repeated failed sign-ins, one source), T1098 account manipulation (new MFA method registered). Write it, run it through `make convert`, check the output looks right.

Linking Cortex to TheHive

One-time: TheHive admin settings, add Cortex as an analyzer server (`http://cortex:9001` inside the Docker network), API key from Cortex's own UI. Once linked, run an analyzer against an observable. Free-tier analyzers that need no external API key are limited, but it proves the wiring works.

IAM attack pattern against LocalStack

```
aws --endpoint-url=http://localhost:4566 iam create-user --user-name shadow-admin
aws --endpoint-url=http://localhost:4566 iam attach-user-policy \
  --user-name shadow-admin --policy-arn arn:aws:iam::aws:policy/AdministratorAccess
aws --endpoint-url=http://localhost:4566 iam create-access-key --user-name shadow-ad
```

Create user, immediately grant admin, immediately mint a key — real privilege-escalation pattern, well documented in the wild. Wrote up what a CloudTrail detection for this would look like even though I can't fire it against real CloudTrail on the free tier (LocalStack Community doesn't emit that log format).

Terraform modules and remote state

The core idea once you're past a single resource: state shouldn't live on one laptop once more than one thing touches it (put it in an Azure Storage blob, locked), and repeated patterns belong in a module instead of copy-pasted blocks.

Where I want to take this next

The real capstone here is wiring the whole chain together — Suricata detects something, Wazuh alerts on it, that alert creates a TheHive case automatically (their REST API supports this), Cortex enriches it the moment it's created. I expect to hit real friction on auth tokens and container networking doing this, and I'm planning to write down each thing that breaks as I go, the same way I've been doing for everything else here.

Also want to actually turn the impossible-travel rule into a real correlation rule instead of the proxy version I have now (which just alerts on Identity Protection's own risk flag rather than computing the travel-distance logic itself). Sigma's correlation spec or Wazuh's own frequency/timeframe rule options can both do this — haven't picked which yet.

Atomic Red Team validation is blocked on me actually building the Windows VM in UTM — deliberately not automated, it's a multi-GB download with a EULA I want to read myself. Once it's up: run a test for T1078.004, check whether the rule I already wrote actually fires.

Adding a GitHub Actions workflow that runs `sigma check / make convert` on every push so a broken rule can't merge silently — small thing, but it's the specific "detection as code" signal a few of the postings I'm looking at called out directly.

Commands I keep forgetting

```
# docker
docker compose up -d / stop / down
docker stats --no-stream
docker system prune

# sigma
export PATH="$HOME/.local/bin:$PATH"
sigma convert -t kusto -p kql-conversions/pipelines/azuread-table-mappings.yml -p se
make convert

# terraform
az login
terraform init / plan / apply / destroy

# localstack
export AWS_ACCESS_KEY_ID=test AWS_SECRET_ACCESS_KEY=test AWS_DEFAULT_REGION=us-east-1
aws --endpoint-url=http://localhost:4566 <command>

# git
git config core.hooksPath .githooks
```

Actual problems I hit (keeping this list honest and specific)

What happened	Why	What fixed it
sigma convert: "unable to determine table name"	<code>sentinel_asim</code> only auto-maps a handful of built-in categories, not Entra ID sign-in/audit logs	Wrote a small pipeline setting <code>query_table</code> explicitly, at a lower priority so it runs first
Wazuh cert generator: permission denied on root-ca-manager.pem	It locks its own output directory read-only before finishing those two files	chmod the dir back open, copy root-ca.pem/key to the manager names by hand
TheHive container exits immediately, "application secret not set"	<code>--no-config-secret</code> doesn't actually disable the check despite the name	Generate a real secret, pass via <code>.env</code> + <code>--secret</code>
LocalStack: no CloudTrail	It's a Pro-only feature	Nothing free — practice IAM/S3/STS instead, or use a real free-tier AWS account

gitleaks missed an obvious secret	Its generic-key rule matches a fixed character set for the value; a <code>\$</code> broke the match	Don't trust it as a guarantee — the real control is never writing real creds to a file to begin with
-----------------------------------	---	--

Before I forget

- 2FA everywhere — GitHub, any cloud account I make for this
- Spending cap set before creating anything on Azure/AWS
- Terraform auth through az login only, never a long-lived key
- `git config core.hooksPath .githooks` on every fresh clone — doesn't happen automatically
- Atomic Red Team: VM only, NAT networking, never bridged, snapshot before every session
- Sanitise screenshots/logs before committing anything
- Change Wazuh's default password
- `terraform destroy` when I'm done, so nothing keeps running